

Задание 1. Создание параллельного приложения

Аннотация: В данном практическом занятии, будет создано простое многопоточное приложение, в котором будет реализовано три различных потока.

1. Создадим простое консольное приложение и назовем его, к примеру, "SimpleMultithreadingApplication":

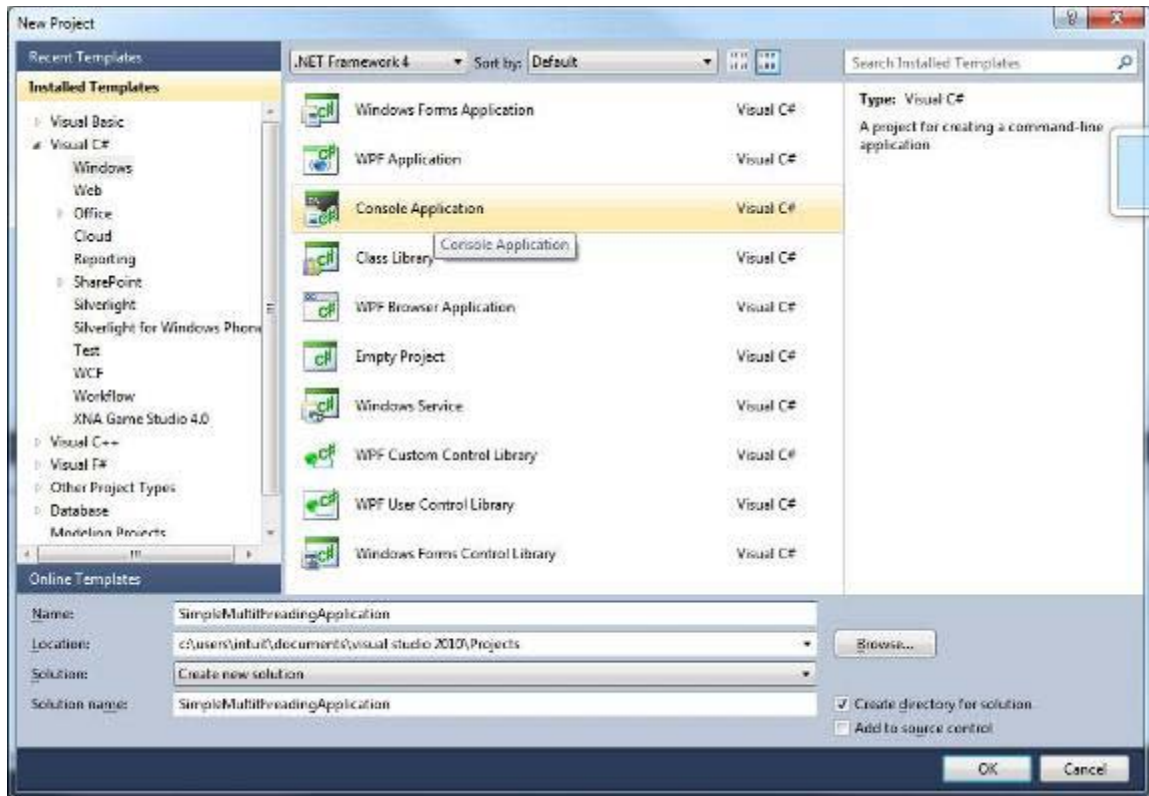


Рис. 1.

2. Далее выведем в однопоточном режиме сообщение "Hello World!" с помощью кода:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
namespace SimpleMultithreadingApplication
{
    class Program
    {
        static void Main(string[] args)
        {
            for (int i = 0; i < 10; i++)
            {
                Console.WriteLine("Hello World!");
            }
            Console.ReadLine();
        }
    }
}
```

Так как, действие выполняется в один поток (метод Main), результат будет следующим:

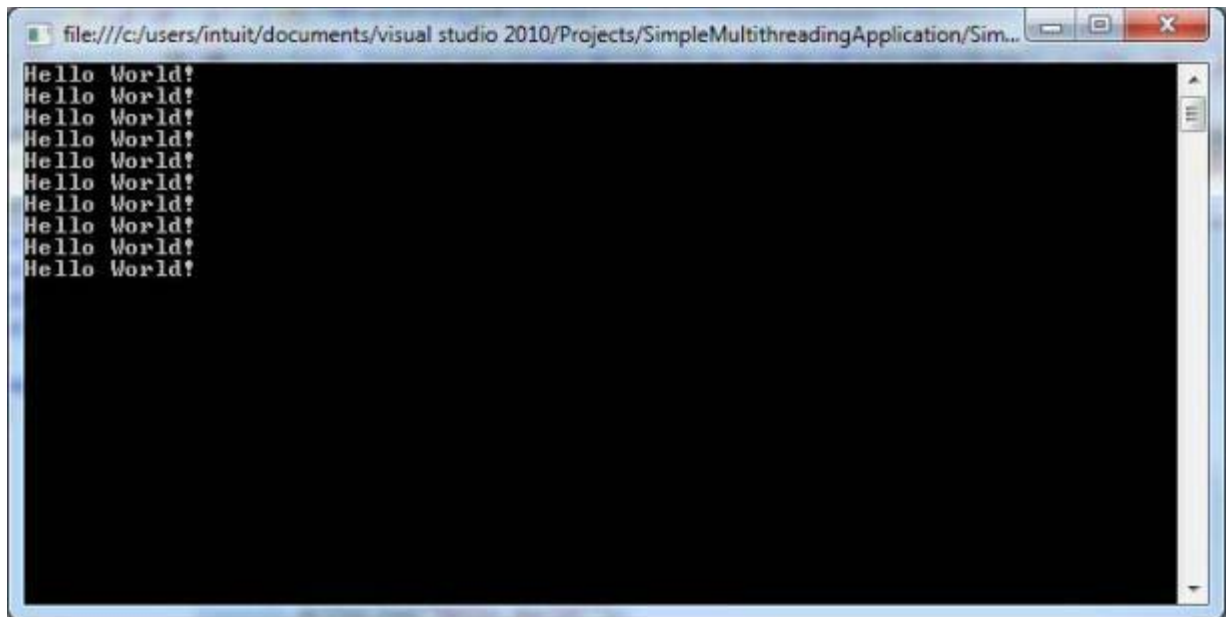


Рис. 2.

3. Для того, что бы использовать многопоточность, используем директиву `System.Threading`:
`using System.Threading;`
4. Теперь, создадим три различных метода, которые будут выполняться в различных потоках:

```
static void FirstThread()
{
    for (int i=0;i<10;i++)
    {
        Console.WriteLine("Первый поток говорит: Hello!");
    }
}
static void SecondThread()
{
    for (int i = 0; i < 10; i++)
    {
        Console.WriteLine("Второй поток говорит: World!");
    }
}
static void ThirdThread()
{
    for (int i = 0; i < 10; i++)
    {
        Console.WriteLine("Третий поток говорит: Hello World!");
    }
}
```

5. В методе `Main` создаем потоки:
`Thread thread1 = new Thread(new ThreadStart(FirstThread))`
`Thread thread2 = new Thread(new ThreadStart(SecondThread))`
`Thread thread3 = new Thread(new ThreadStart(ThirdThread))`

В нашем случае поток:

- o `thread1` - вызывает метод `FirstThread`
- o `thread2` - вызывает метод `SecondThread`

- o thread3 - вызывает метод ThirdThread
- 6. Запускаем потоки, с помощью метода Start():


```
thread1.Start()
thread2.Start()
thread3.Start()
```
- 7. Добавим на вывод в главном потоке (метод Main), следующие строки:


```
Console.WriteLine("Главный поток молчит")
Console.WriteLine("Завершение главного потока")
Console.ReadLine()
```

Весь код будет выглядеть следующим образом:

```
static void Main(string[] args)
{
    Thread thread1 = new Thread(new ThreadStart(FirstThread));
    Thread thread2 = new Thread(new
ThreadStart(SecondThread));
    Thread thread3 = new Thread(new ThreadStart(ThirdThread));
    thread1.Start();
    thread2.Start();
    thread3.Start();
    Console.WriteLine("Главный поток молчит");
    Console.WriteLine("Завершение главного потока");
    Console.ReadLine();
}
```

- 8. Запустим приложение. Результат будет следующим:

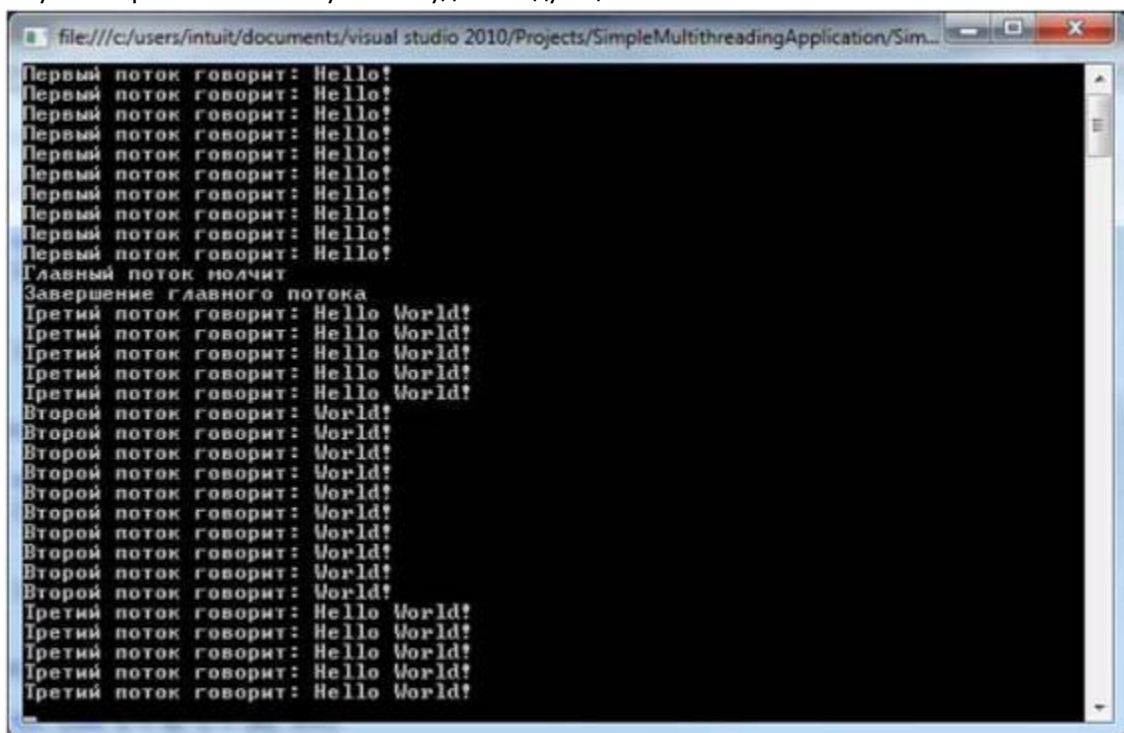


Рис. 3.

Как видно из результата, потоки выполняются с различным интервалом по времени.

- 9. Для того, что бы задать время блокировки, на выполнение потока, используем метод Sleep(). Для главного потока установим время блокировки 1000мс:

```
Thread.Sleep(1000)
Console.WriteLine("Главный поток молчит")
Console.WriteLine("Завершение главного потока")
Console.ReadLine()
```

10. Аналогично, установим время блокировки, для дочерних потоков 20мс, 100мс, 90мс:

```
static void FirstThread()
{
    for (int i=0;i<10;i++)
    {
        Thread.Sleep(20);
        Console.WriteLine("Первый поток говорит: Hello!");
    }
    Console.WriteLine("Завершение первого потока");
}
static void SecondThread()
{
    for (int i = 0; i < 10; i++)
    {
        Thread.Sleep(100);
        Console.WriteLine("Второй поток говорит: World!");
    }
    Console.WriteLine("Завершение второго потока");
}
static void ThirdThread()
{
    for (int i = 0; i < 10; i++)
    {
        Thread.Sleep(90);
        Console.WriteLine("Третий поток говорит: Hello World!");
    }
    Console.WriteLine("Завершение третьего потока");
}
```

11. Запустим программу. Результат будет следующим:

